

企业知识库 Agent 助手项目完整帮助文档

基于当前项目代码实时更新

2026-05-09

目录

1	项目一句话说明	4
2	项目能做什么	4
3	真实调用链路	4
4	目录结构	5
5	环境要求与安装	6
6	配置 DashScope API Key	6
7	核心配置说明	7
7.1	模型配置	7
7.2	向量库配置	7
7.3	Prompt 配置	8
7.4	员工模拟数据配置	8
8	核心知识点	8
8.1	什么是 RAG	8
8.2	为什么要切分文档	9
8.3	什么是 Chroma	9
8.4	什么是 ReAct Agent	9

目录	2
9 Agent 工具说明	9
10 主要工作流程	10
10.1 企业制度问答	10
10.2 员工个人数据查询	10
10.3 个人周报生成	11
10.4 知识文档上传和入库	11
11 启动应用	11
12 推荐测试问题	11
13 主要代码入口	12
13.1 app.py	12
13.2 agent/react_agent.py	12
13.3 agent/tools/agent_tools.py	12
13.4 rag/vector_store.py	13
13.5 rag/rag_service.py	13
13.6 model/factory.py	13
14 如何新增知识库文档	13
15 如何新增 Agent 工具	14
16 常见问题	14
16.1 启动时报 DashScope API Key 错误	14
16.2 问答结果和知识库内容无关	15
16.3 新增文档后没有被检索到	15
16.4 Agent 输出格式错误或中断	15
16.5 中文显示乱码	15
17 学习路线	15
18 简历写法参考	16

目录	3
19 验收问题与答案	16
19.1 制度类问题什么时候调用 <code>rag_summarize</code>	16
19.2 个人数据类问题为什么要先获取员工 ID	16
19.3 周报生成需要哪些工具	17
19.4 如何让新增制度文档生效	17
20 一键运行流程	17
21 注意事项	17
22 参考资料	17

1 项目一句话说明

这是一个基于 Streamlit + LangChain ReAct Agent + Chroma + DashScope 的企业内部知识库智能助手示例项目。项目面向 AI 应用开发学习、简历展示和企业知识问答演示，支持企业制度问答、员工模拟数据查询、个人周报生成、知识文档上传入库和流式对话展示。

当前项目已经从早期的产品客服场景迁移为企业知识库场景。知识库资料位于 `data/enterprise/`，结构化员工模拟数据位于 `data/enterprise_external/employee_records.csv`，主界面入口是 `app.py`。

2 项目能做什么

当前项目主要支持以下能力：

1. 企业制度知识库问答：基于 HR、财务、IT 支持、项目协作、信息安全、入职指引等制度文档进行 RAG 检索增强回答。
2. ReAct Agent 工具调度：Agent 根据用户问题自动选择知识库检索、员工 ID 获取、部门查询、员工记录查询、周报上下文生成等工具。
3. 员工模拟数据查询：支持查询年假余额、报销状态、项目记录、入职待办和周报相关业务数据。
4. 个人周报生成：按固定工具流程获取员工 ID、部门、周报上下文和项目记录，再生成结构化中文周报。
5. Streamlit 对话界面：提供类聊天体验、流式输出、检索来源折叠展示、示例问题入口、模型和检索参数控制。
6. 知识库维护：侧边栏可上传 TXT/PDF 文档，保存到 `data/enterprise/` 并写入 Chroma 向量库。
7. 会话导出：可将当前对话导出为 Markdown 文件。

3 真实调用链路

```
Streamlit(app.py)
-> ReactAgent(agent/react_agent.py)
-> AgentExecutor.stream()
-> Tools(agent/tools/agent_tools.py)
-> RagSummarizeService(rag/rag_service.py)
-> VectorStoreService(rag/vector_store.py)
-> Chroma(chroma_db/)
-> DashScope(model/factory.py)
-> Prompt(prompts/*.txt)
```

用户在 `app.py` 的 `st.chat_input()` 输入问题后,前端会调用 `ReactAgent.execute_stream()`。
Agent 内部通过 `create_react_agent()` 创建 ReAct Agent,再由 `AgentExecutor` 执行工具
调用循环。最终答案通过 `AgentExecutor.stream()` 分块返回给 `Streamlit`,并由 `st.write_stream()`
展示。

4 目录结构

```
.
|-- app.py                # Streamlit 应用入口
|-- requirements.txt      # Python 运行依赖
|-- help_requirements.txt # 帮助文档生成相关依赖
|-- HELP_PROJECT_GUIDE.tex # 当前帮助文档 LaTeX 源文件
|-- HELP_PROJECT_GUIDE.pdf # 当前帮助文档 PDF
|-- build_help_pdf.py     # 旧版 Markdown 到 LaTeX 辅助脚本
|-- build_help_pdf_reportlab.py # 旧版 ReportLab PDF 辅助脚本
|-- md5_enterprise.txt    # 已入库知识文件 MD5 记录
|-- config/
|   |-- agent.yml        # 员工模拟数据路径配置
|   |-- chroma.yml       # Chroma 与知识库入库配置
|   |-- prompts.yml      # Prompt 文件路径配置
|   `-- rag.yml          # DashScope 模型配置
|-- agent/
|   |-- react_agent.py    # ReAct Agent 创建与流式执行逻辑
|   `-- tools/
|       |-- agent_tools.py # 企业场景工具函数
|       `-- middleware.py  # 中间件示例代码
|-- rag/
|   |-- vector_store.py   # 文档加载、切分、向量入库、Retriever 创建
|   `-- rag_service.py    # RAG 检索与总结链路
|-- model/
|   `-- factory.py        # ChatTongyi 与 DashScope Embeddings 初始化
|-- prompts/
|   |-- main_prompt.txt   # Agent 主 Prompt
|   |-- rag_summarize.txt # RAG 总结 Prompt
|   `-- report_prompt.txt # 报告生成 Prompt
|-- data/
|   |-- enterprise/      # 企业制度知识库文档
|   |   |-- finance_reimbursement_policy.txt
|   |   |-- hr_leave_policy.txt
|   |   |-- it_support_policy.txt
|   |   |-- onboarding_guide.txt
|   |   |-- project_collaboration_policy.txt
|   |   `-- security_policy.txt
```

```
|  `-- enterprise_external/
|      `-- employee_records.csv  # 员工模拟结构化数据
|-- utils/                      # 配置、路径、文件、日志工具
|-- chroma_db/                  # Chroma 本地持久化目录
`-- logs/                       # 运行日志目录
```

5 环境要求与安装

运行项目前需要准备：

- Python 3.10 或更高版本。
- 可用的阿里云 DashScope API Key。
- Windows PowerShell、CMD、macOS 或 Linux Shell 均可运行。

安装依赖：

```
pip install -r requirements.txt
```

如果下载较慢，可使用清华源：

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

当前 requirements.txt 中的关键依赖包括：

```
langchain==0.3.7
langchain-core==0.3.18
langchain-community==0.3.7
langchain-chroma==0.1.4
chromadb==0.5.15
langchain-text-splitters==0.3.2
langgraph==0.2.50
streamlit==1.40.1
pypdf==5.1.0
pyyaml==6.0.2
dashscope==1.20.14
```

6 配置 DashScope API Key

项目通过 DashScope 调用通义千问聊天模型和 Embedding 模型，运行前需要配置 DASHSCOPE_API_KEY。

PowerShell 临时配置：

```
$env:DASHSCOPE_API_KEY="your-api-key"
```

CMD 临时配置:

```
set DASHSCOPE_API_KEY=your-api-key
```

macOS/Linux 临时配置:

```
export DASHSCOPE_API_KEY="your-api-key"
```

7 核心配置说明

7.1 模型配置

文件: config/rag.yml

```
chat_model_name: qwen3-max  
embedding_model_name: text-embedding-v4
```

- chat_model_name: 聊天模型名称, 当前使用 qwen3-max。
- embedding_model_name: 向量化模型名称, 当前使用 text-embedding-v4。

7.2 向量库配置

文件: config/chroma.yml

```
collection_name : enterprise_agent  
persist_directory : chroma_db  
k : 3  
data_path : data/enterprise  
md5_hex_store : md5_enterprise.txt  
allow_knowledge_file_type : ["txt","pdf"]  
chunk_size : 200  
chunk_overlap : 20
```

- collection_name: Chroma 集合名称, 当前为 enterprise_agent。
- persist_directory: 向量库持久化目录, 当前为 chroma_db。
- k: 每次检索返回的 top-k 文档片段数, 当前为 3。
- data_path: 知识库文档目录, 当前为 data/enterprise。
- md5_hex_store: 已处理文件 MD5 记录文件, 当前为 md5_enterprise.txt。
- allow_knowledge_file_type: 允许入库的文档类型, 当前支持 txt 和 pdf。
- chunk_size 和 chunk_overlap: 控制文档切分粒度和相邻片段重叠长度。

7.3 Prompt 配置

文件: config/prompts.yml

```
main_prompt_path : prompts/main_prompt.txt
rag_summarize_prompt_path : prompts/rag_summarize.txt
report_prompt_path : prompts/report_prompt.txt
```

- main_prompt.txt: Agent 主提示词, 定义企业知识库助手角色、工具边界和固定 workflow。
- rag_summarize.txt: RAG 检索后的总结提示词, 要求答案基于参考资料。
- report_prompt.txt: 报告生成提示词, 目前保留为报告场景扩展入口。

7.4 员工模拟数据配置

文件: config/agent.yml

```
external_data_path: data/enterprise_external/employee_records.csv
```

employee_records.csv 模拟企业业务系统中的员工数据, 字段包括:

```
employee_id, employee_name, department, leave_balance_days,
reimbursement_status, missing_reimbursement_docs, project_name,
weekly_highlights, deliverables, next_week_plan, risks, onboarding_tasks
```

8 核心知识点

8.1 什么是 RAG

RAG 是 Retrieval-Augmented Generation, 即检索增强生成。它先从外部知识库检索和问题相关的资料, 再把资料连同用户问题一起交给大模型生成答案。

本项目中的 RAG 链路是:

用户问题 -> Chroma 检索企业制度片段 -> 拼接 context -> DashScope 模型总结回答

它解决的问题是: 大模型自身不知道本地企业制度文档、流程规范和模拟业务资料, 所以需要先检索, 再回答。

8.2 为什么要切分文档

原始制度文档可能较长，直接整体向量化会降低检索粒度。切分为 chunk 后，每个片段都可以单独向量化和检索，更容易命中具体条款、流程步骤或注意事项。

8.3 什么是 Chroma

Chroma 是本地向量数据库,用来存储文档片段、向量和元数据。项目使用 `langchain_chroma.Chroma` 创建本地持久化向量库:

```
Chroma(  
    collection_name="enterprise_agent",  
    embedding_function=embed_model,  
    persist_directory="chroma_db",  
)
```

8.4 什么是 ReAct Agent

ReAct 是 Reasoning + Acting。模型按照“思考、选择工具、读取观察结果、继续推理、最终回答”的循环工作。普通 LLM 调用只生成文本，ReAct Agent 可以根据问题选择工具并把工具结果整合成最终答案。

本项目中的 Agent 使用以下标准格式:

Thought -> Action -> Action Input -> Observation -> Final Answer

9 Agent 工具说明

当前 `agent/react_agent.py` 注册的工具有:

```
tools = [  
    rag_summarize,  
    get_employee_id,  
    get_employee_department,  
    fetch_employee_records,  
    generate_weekly_report_context,  
]
```

工具	用途
----	----

<code>rag_summarize</code>	从企业知识库检索制度、流程、IT 支持、项目规范、信息安全等参考资料，并生成基于资料的回答。
<code>get_employee_id</code>	获取当前员工 ID。当前为演示逻辑，会从 E1001 到 E1005 中随机返回一个。
<code>get_employee_department</code>	根据员工 ID 查询员工所属部门。
<code>fetch_employee_records</code>	查询员工模拟业务数据，入参格式为 员工 ID，查询类型。查询类型包括 <code>leave_balance</code> 、 <code>reimbursement</code> 、 <code>projects</code> 、 <code>onboarding</code> 、 <code>weekly_report</code> 、 <code>all</code> 。
<code>generate_weekly_report_context</code>	进入员工周报生成场景，提示 Agent 输出本周概览、关键产出、问题风险和下周计划。

10 主要工作流程

10.1 企业制度问答

用户询问制度、流程或规范

```
-> Streamlit 调用 ReactAgent.execute_stream()
-> Agent 判断需要企业知识库资料
-> 调用 rag_summarize
-> RagSummarizeService 检索 Chroma
-> 拼接参考资料 context
-> DashScope 模型基于资料生成回答
-> Streamlit 流式展示答案并折叠展示检索来源
```

10.2 员工个人数据查询

当用户询问“我的年假还有多少”“我的报销状态如何”“我有哪些入职待办”等个人数据问题时，Agent 应按以下流程：

1. 如果用户没有显式提供员工 ID，先调用 `get_employee_id`。
2. 调用 `fetch_employee_records`，入参为 员工 ID，查询类型。
3. 只根据工具返回的模拟结构化数据回答，不编造额外状态。

常见查询类型：

```
leave_balance    # 年假余额
reimbursement    # 报销状态
projects         # 项目记录
onboarding       # 入职待办
weekly_report    # 周报数据
all              # 返回完整记录
```

10.3 个人周报生成

周报生成在 `agent/react_agent.py` 中被写入固定流程：

1. 调用 `get_employee_id` 获取员工 ID。
2. 调用 `get_employee_department` 获取部门。
3. 调用 `generate_weekly_report_context` 进入周报生成场景。
4. 调用 `fetch_employee_records`，入参为 `employee_id, weekly_report`。
5. 用中文生成结构化周报。

10.4 知识文档上传和入库

Streamlit 侧边栏提供 TXT/PDF 上传入口。点击“保存并载入知识库”后，文件会保存到 `data/enterprise/`，然后调用：

```
rag_service.vector_store.load_document()
```

入库过程会扫描 `data/enterprise/`，按 `config/chroma.yml` 的配置读取 TXT/PDF，计算 MD5 去重，切分文档，生成 Embedding，并写入 `chroma_db/`。

11 启动应用

在项目根目录执行：

```
streamlit run app.py
```

启动成功后，终端通常会显示：

```
Local URL: http://localhost:8501
```

浏览器访问该地址即可使用企业知识库问答界面。

12 推荐测试问题

企业制度问答：

差旅报销需要哪些材料？

公司对外部 AI 工具使用有什么安全要求？

VPN 无法连接应该怎么处理？

新员工入职第一周应该完成哪些事项？

员工数据查询：

我还剩多少年假？

我的报销状态怎么样？

我有哪些入职待办？

我本周参与了哪些项目工作？

周报生成：

根据我的本周项目记录生成周报。

帮我整理一份个人本周工作总结。

13 主要代码入口

13.1 app.py

app.py 是 Streamlit 前端入口，主要职责包括：

- 初始化 `ReactAgent` 和会话状态。
- 注入自定义 CSS，构建企业知识库对话界面。
- 在侧边栏提供模型选择、Temperature、Top-K 检索数量、文件上传、会话导出和清空会话。
- 接收用户输入，调用 `stream_agent_response()`。
- 预先检索来源文档，并在助手回复后以折叠面板展示引用片段。
- 将完整助手回复通过 `"".join(response_chunks)` 保存到会话历史。

13.2 agent/react_agent.py

负责构建 ReAct Agent：

- 读取 `prompts/main_prompt.txt`。
- 注册企业场景工具列表。
- 拼接 ReAct 标准模板和周报、员工数据固定流程。
- 使用 `create_react_agent()` 创建 Agent。
- 使用 `AgentExecutor` 包装执行，并启用 `handle_parsing_errors`。
- 通过 `execute_stream()` 提供流式输出。

13.3 agent/tools/agent_tools.py

定义 Agent 可调用工具。该文件会初始化 `RagSummarizeService`，并通过 `@tool` 暴露企业知识检索、员工 ID、部门、员工记录和周报上下文工具。

13.4 rag/vector_store.py

负责知识库入库和 retriever 创建：

- 初始化 Chroma。
- 使用 RecursiveCharacterTextSplitter 切分文档。
- 扫描 data/enterprise/ 中允许类型的文档。
- 用 MD5 记录避免重复入库。
- 通过 add_documents() 写入向量库。
- 返回 as_retriever(search_kwargs={"k": 3})。

13.5 rag/rag_service.py

负责 RAG 问答链路：

- 调用 retriever 检索相关文档。
- 拼接参考资料 context。
- 使用 rag_summarize.txt 限制模型基于资料回答。
- 通过 StrOutputParser() 输出纯字符串。

13.6 model/factory.py

负责模型初始化：

- ChatTongyi(model=rag_conf["chat_model_name"])：聊天模型。
- DashScopeEmbeddings(model=rag_conf["embedding_model_name"])：Embedding 模型。

14 如何新增知识库文档

1. 将 .txt 或 .pdf 文件放入 data/enterprise/。
2. 确认 config/chroma.yml 中允许该文件类型。
3. 执行知识库入库命令：

```
python -m rag.vector_store
```

也可以在 Streamlit 侧边栏上传 TXT/PDF 后点击“保存并载入知识库”。

建议：

- TXT 文件使用 UTF-8 编码。
- 文档内容按主题分段，便于切分和检索。

- 如果问题经常检索不到，可适当调大 Top-K，或优化知识库文档表达。
- 如果文档改动后没有重新入库，检查 md5_enterprise.txt 和 logs/。
- 如果确实需要完全重建向量库，请手动确认旧向量库和 MD5 记录的处理方式，不要使用批量递归删除命令。

15 如何新增 Agent 工具

新增工具通常需要修改两处：agent/tools/agent_tools.py 和 agent/react_agent.py。

第一步，在 agent/tools/agent_tools.py 中定义工具：

```
from langchain_core.tools import tool

@tool
def get_policy_owner(policy_name: str) -> str:
    """ 根据制度名称查询制度负责人。入参为制度名称字符串。"""
    return "HR"
```

第二步，在 agent/react_agent.py 中导入并加入 tools 列表：

```
tools = [
    rag_summarize,
    get_employee_id,
    get_policy_owner,
]
```

第三步，在 prompts/main_prompt.txt 中补充工具调用场景和入参格式。工具 docstring 和 Prompt 描述越清楚，Agent 越容易正确调用。

16 常见问题

16.1 启动时报 DashScope API Key 错误

检查是否设置了环境变量：

```
$env:DASHSCOPE_API_KEY
```

如果没有输出，重新设置：

```
$env:DASHSCOPE_API_KEY="your-api-key"
```

16.2 问答结果和知识库内容无关

建议检查：

- `chroma_db/` 是否已经生成。
- 是否执行过 `python -m rag.vector_store`。
- `data/enterprise/` 中是否有相关制度文档。
- `config/chroma.yml` 中的 `k` 是否过小。
- 文档编码是否为 UTF-8。
- `rag_summarize.txt` 是否仍要求模型基于参考资料回答。

16.3 新增文档后没有被检索到

项目使用 MD5 记录已处理文件。如果文件内容没有变化，再次运行入库时会跳过相同 MD5 的文件。可以查看 `md5_enterprise.txt` 和 `logs/` 确认是否被跳过。

16.4 Agent 输出格式错误或中断

ReAct Agent 对输出格式较严格。项目已在 `AgentExecutor` 中配置 `handle_parsing_errors`。如果仍失败，可以：

- 换一种更明确的问题问法。
- 检查 `prompts/main_prompt.txt` 是否被改坏。
- 换用更稳定的聊天模型。
- 减少 Prompt 中和 ReAct 标准格式冲突的说明。

16.5 中文显示乱码

如果 Windows 终端中显示乱码，优先用支持 UTF-8 的编辑器查看文件。LaTeX 源文件应保存为 UTF-8，并使用支持中文的编译方式生成 PDF。

17 学习路线

建议按以下顺序阅读代码：

1. 先看 `app.py`，理解前端如何接收问题、展示答案、上传文档和导出会话。
2. 再看 `agent/react_agent.py`，理解 Agent 如何创建、如何注册工具、如何约束周报流程。
3. 再看 `agent/tools/agent_tools.py`，理解每个工具具体做什么。
4. 再看 `rag/vector_store.py`，理解文档如何入库。

5. 再看 `rag/rag_service.py`，理解检索结果如何交给模型总结。
6. 最后看 `prompts/*.txt` 和 `config/*.yaml`，理解配置和提示词如何影响效果。

18 简历写法参考

可以写成：

基于 LangChain + ReAct Agent + Chroma 构建企业知识库智能助手，支持企业制度问答、员工模拟数据查询和自动周报生成。系统通过 RAG 检索内部制度文档，结合 ReAct Agent 动态调用员工信息、报销记录、请假余额、项目进展等工具，并使用 Streamlit 实现流式对话、检索来源展示、知识文档上传和会话导出。

项目亮点：

- 构建企业知识库 RAG 流程，支持 TXT/PDF 文档切分、向量化入库、top-k 检索和基于上下文的回答生成。
- 基于 ReAct Agent 注册多类业务工具，实现制度问答、员工信息查询、项目记录查询和周报生成等多任务自动调度。
- 使用 YAML 管理模型、Prompt、向量库和业务数据路径，提高项目可配置性和可迁移性。
- 通过模拟企业数据集实现“知识库 + 结构化数据”的混合问答，增强项目业务完整度。
- 构建 Streamlit 对话界面，支持模型选择、Temperature、Top-K、文件上传入库、引用来源展示和会话导出。

19 验收问题与答案

19.1 制度类问题什么时候调用 `rag_summarize`

当用户询问 HR、财务报销、IT 支持、项目协作、信息安全、入职指引等公共制度或流程问题时，Agent 应优先调用 `rag_summarize`，从企业知识库中获取依据后再回答。

19.2 个人数据类问题为什么要先获取员工 ID

因为年假余额、报销状态、项目记录、入职待办等数据都和具体员工绑定。用户问“我”的状态时，Agent 需要先调用 `get_employee_id`，再用 `fetch_employee_records` 查询模拟业务数据。

19.3 周报生成需要哪些工具

周报生成需要依次调用 `get_employee_id`、`get_employee_department`、`generate_weekly_report_content`、`fetch_employee_records`，最后根据 `weekly_report` 数据输出中文周报。

19.4 如何让新增制度文档生效

把 TXT/PDF 放入 `data/enterprise/` 后执行 `python -m rag.vector_store`，或在 Streamlit 侧边栏上传文档并点击保存入库。入库成功后，新的制度内容会进入 Chroma 检索范围。

20 一键运行流程

首次运行：

```
cd D:\my_develop\A_work_program\LangChain-ReAct-Agent-main
pip install -r requirements.txt
$env:DASHSCOPE_API_KEY="your-api-key"
python -m rag.vector_store
streamlit run app.py
```

后续运行：

```
cd D:\my_develop\A_work_program\LangChain-ReAct-Agent-main
$env:DASHSCOPE_API_KEY="your-api-key"
streamlit run app.py
```

21 注意事项

- 当前员工数据是模拟数据，不接入真实公司系统。
- `get_employee_id` 当前随机返回员工 ID，适合演示，不适合生产身份识别。
- 项目用于学习、演示和简历展示时，不要放入真实员工隐私、客户信息、合同、源代码或其他敏感数据。
- 如需完全重建向量库，请手动确认要处理的文件，不要使用批量递归删除命令。
- `HELP.md` 当前不在项目中，本帮助文档直接维护 `HELP_PROJECT_GUIDE.tex`，避免旧生成脚本重新写入过期内容。

22 参考资料

- LangChain ReAct Agent API: <https://reference.langchain.com/python/langchain-classic/>

- LangChain AgentExecutor API:<https://reference.langchain.com/python/langchain-classic>
- LangChain Tools 文档:<https://docs.langchain.com/oss/python/langchain-tools>
- LangChain Chroma 集成文档:<https://docs.langchain.com/oss/python/integrations/vector>
- Chroma 官方文档: <https://docs.trychroma.com/docs/overview/getting-started>
- Streamlit Chat Elements 文档:<https://docs.streamlit.io/library/api-reference/chat>
- LangChain ChatTongyi 文档:<https://docs.langchain.com/oss/python/integrations/chat/to>
- LangChain DashScope Embeddings 文档:<https://docs.langchain.com/oss/python/integratio>